

Data Analytics Internship Program 2024

Final Project Presentation

Project Name : India's Silent Epidemic: A Data-Driven Analysis of Suicide Patterns
Unique ID : IBM1428
Team Name : GPG Girls
College Name : Government Polytechnic For Girls, Gujarat

Team Members

Sr. No.	NAME
1	Bhanushali Priyanshi
2	Gohel Durva
3	Joshi Dhruvi
4	Kotecha Mishri
5	Patel Khiloni
6	Patel Sneha
7	Radadiya Mahi
8	Vaghasiya Tanvi

Introduction

● Overview of the Project

Suicide is a critical public health issue in India, with significant socio-economic and demographic implications. This project aims to analyse and predict suicide rates in India from 2001 to 2012 using machine learning techniques. By leveraging data analysis tools, the project seeks to inform evidence-based prevention strategies and propose actionable interventions to reduce suicide rates for the overall well-being of the population.



● Objective

Specific objectives include:

- To analyse historical suicide data from 2001 to 2012 in India.
- To identify key factors like demographic, geographic, and socio-economic of suicide rates.
- To analyse temporal trends in suicide rates to identify potential risk periods.
- To develop a predictive model with an accuracy of 0.82 to forecast future suicide trends.
- To provide evidence-based prevention strategies and recommendations for policy makers and public health officials.

Problem Identification



1

Problem Statement

Suicides in India pose a severe challenge, exacerbated by factors such as mental health issues, economic distress, and societal pressures. Despite various efforts, suicide rates remain high due to the lack of precise predictive models and targeted intervention strategies. This project seeks to address this issue by analyzing historical data and developing a machine learning model to predict suicide trends for understanding the complex factors influencing suicide.



2

Significance of the Problem

India grapples with a silent epidemic of suicide, a crisis impacting individuals, families, and society at large. The underreporting of this issue shrouds its true magnitude, necessitating a robust data-driven approach to inform comprehensive prevention strategies. and factors contributing to suicides in India. This knowledge is crucial for developing effective prevention strategies.



3

Relevant SDGs

SDG 3: Good Health and Well Being

Data Collection

Sources of Data

The project will utilize datasets from the following sources:

- Dataset: Suicides in India 2001-2012.csv
- Sources:
 - Government health departments
 - Public records
 - Research institutions.

- **Data Description**

- **Data Collection Methods**

Data Preprocessing

● Data Cleaning Methods

9.1 Label encoding on the dataset

```
In [49]: # Encode categorical variables
label_encoders = {}
for column in ['State', 'Type_code', 'Type', 'Gender', 'Age_group', 'Total_binned']:
    label_encoders[column] = LabelEncoder()
    df[column] = label_encoders[column].fit_transform(df[column])
```

```
In [50]: # Create new features
df['State_Type'] = df['State'].astype(str) + '_' + df['Type'].astype(str)
df['Gender_Age'] = df['Gender'].astype(str) + '_' + df['Age_group'].astype(str)

# Encode the new features
df['State_Type'] = label_encoders['State'].fit_transform(df['State_Type'])
df['Gender_Age'] = label_encoders['Gender'].fit_transform(df['Gender_Age'])
```

9.2 Set features and targets

```
In [51]: # Features and target
features = df.drop(columns=['Total', 'Total_binned'])
target = df['Total_binned']
```

```
In [52]: # Scale numerical features
scaler = StandardScaler()
features[['Year']] = scaler.fit_transform(features[['Year']])
```

● Handling Missing Values

```
In [6]: #checking for null values
df.isnull().sum()
```

```
Out[6]: State          0
Year          0
Type_code     0
Type          0
Gender        0
Age_group     0
Total         0
dtype: int64
```

Nice! There are no NaN values.

```
In [7]: # checking for duplicates
df.duplicated().sum()
```

```
Out[7]: 0
```

No Duplicates as well!

Data Preprocessing - Data Transformation Techniques

9.4 Model Training

```
# Grid search for GBM
grid_gbm = GridSearchCV(HistGradientBoostingClassifier(), param_grid_gbm, cv=5, n_jobs=-1, verbose=1)
grid_gbm.fit(X_train, y_train)
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

```
>
  GridSearchCV
  estimator: HistGradientBoostingClassifier
    HistGradientBoostingClassifier
```

```
# Best parameters and score for GBM
best_gbm = grid_gbm.best_estimator_
print("Best parameters for GBM:", grid_gbm.best_params_)
print("Best cross-validation score for GBM:", grid_gbm.best_score_)
```

Best parameters for GBM: {'learning_rate': 0.1, 'max_depth': 7, 'max_iter': 100, 'max_leaf_nodes': 63}
Best cross-validation score for GBM: 0.6612031329958181

```
# Train the GBM model
gbm = HistGradientBoostingClassifier()
gbm.fit(X_train, y_train)
```

```
> HistGradientBoostingClassifier
HistGradientBoostingClassifier()
```

```
# Train the best GBM model
best_gbm.fit(X_train, y_train)
```

```
>
  HistGradientBoostingClassifier
HistGradientBoostingClassifier(max_depth=7, max_leaf_nodes=63)
```

```
# Evaluate the best GBM model
y_pred_gbm = best_gbm.predict(X_test)
accuracy_gbm = accuracy_score(y_test, y_pred_gbm)
print(f"Best GBM Accuracy: {accuracy_gbm:.2f}")

Best GBM Accuracy: 0.68
```

The accuracy of the model is 68 % let's train it further

```
print("Best GBM Classification Report:")
print(classification_report(y_test, y_pred_gbm))
```

Best GBM Classification Report:

	precision	recall	f1-score	support
0	0.70	0.97	0.82	16344
1	0.52	0.24	0.33	7425
2	0.72	0.67	0.69	3356
3	0.80	0.86	0.83	257
4	0.53	0.07	0.13	2337
5	0.68	0.45	0.54	557
6	0.25	0.94	0.40	17
accuracy			0.68	30333
macro avg	0.60	0.60	0.53	30333
weighted avg	0.65	0.68	0.63	30333

```
print("Best GBM Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_gbm))
```

Best GBM Confusion Matrix:

```
[[15900 368 45 5 8 5 13]
 [ 5355 1814 156 0 85 2 13]
 [ 479 486 2257 7 54 73 0]
 [ 2 0 10 254 0 30 1]
 [ 895 809 432 5 169 6 21]
 [ 11 5 243 46 0 252 0]
 [ 0 0 0 1 0 0 16]]
```

```
# Define the parameter grid for XGBoost
param_grid_xgb = {
    'learning_rate': [0.01, 0.1, 0.2],
    'n_estimators': [100, 200],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
```

9.5 Model Training with XGBoost

```
# Grid search for XGBoost
grid_xgb = GridSearchCV(xgb.XGBClassifier(), param_grid_xgb, cv=5, n_jobs=-1, verbose=1)
grid_xgb.fit(X_train, y_train)
```

Fitting 5 folds for each of 72 candidates, totalling 360 fits

```
>
  GridSearchCV
  estimator: XGBClassifier
    XGBClassifier
```

```
# Best parameters and score for XGBoost
best_xgb = grid_xgb.best_estimator_
print("Best parameters for XGBoost:", grid_xgb.best_params_)
print("Best cross-validation score for XGBoost:", grid_xgb.best_score_)
```

Best parameters for XGBoost: {'colsample_bytree': 1.0, 'learning_rate': 0.2, 'max_depth': 7, 'n_estimators': 200, 'subsample': 0.8}
Best cross-validation score for XGBoost: 0.812622748644588

```
# Train the best XGBoost model
best_xgb.fit(X_train, y_train)
```

```
>
  XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
  colsample_bylevel=None, colsample_bynode=None,
  colsample_bytree=1.0, device=None, early_stopping_rounds=None,
  enable_categorical=False, eval_metric=None, feature_types=None,
  gamma=None, grow_policy=None, importance_type=None,
  interaction_constraints=None, learning_rate=0.2, max_bin=None,
  max_cat_threshold=None, max_cat_to_onehot=None,
  max_delta_step=None, max_depth=7, max_leaves=None,
  min_child_weight=None, missing=nan, monotone_constraints=None,
  multi_strategy=None, n_estimators=200, n_jobs=None,
  num_parallel_tree=None, objective='multi:softprob', ...)
```

Data Preprocessing

● Data Transformation Techniques

```
# Evaluate the best XGBoost model
y_pred_xgb = best_xgb.predict(X_test)
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
print(f"Best XGBoost Accuracy: {accuracy_xgb:.2f}")
```

Best XGBoost Accuracy: 0.82

```
print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

Best XGBoost Classification Report:

	precision	recall	f1-score	support	0	0.88	0.93	0.91	16344	1	0.71	0.68	0.70	7425	2	0.82	0.87	0.84	3356	3	0.93	0.91	0.92	297	4		
0.62	0.41	0.50	2337	5	0.78	0.74	0.76	557	6	1.00	0.94	0.97	17	n	n	accuracy	0.82	30333	macro avg	0.82	0.78	0.80	30333	n	weighted avg	0.81	0.82
0.81	30333	n																									

```
print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

Best XGBoost Classification Report:

	precision	recall	f1-score	support	0	0.88	0.93	0.91	16344	1	0.71	0.68	0.70	7425	2	0.82	0.87	0.84	3356	3	0.93	0.91	0.92	297	4		
0.62	0.41	0.50	2337	5	0.78	0.74	0.76	557	6	1.00	0.94	0.97	17	n	n	accuracy	0.82	30333	macro avg	0.82	0.78	0.80	30333	n	weighted avg	0.81	0.82
0.81	30333	n																									

Lastly Generate a pandas profiling for even better visualization

```
from pandas_profiling import ProfileReport
```

```
# Generate the report
```

```
profile = ProfileReport(df, title="Suicides in India Dataset Profiling Report")
profile.to_file("suicides_in_india_profile_report.html")
```

```
# Display the report
```

```
profile.to_notebook_iframe()
```

```
Summarize dataset: 0% | 0/5 [00:00<?, ?it/s]
Generate report structure: 0% | 0/1 [00:00<?, ?it/s]
Render HTML: 0% | 0/1 [00:00<?, ?it/s]
Export report to file: 0% | 0/1 [00:00<?, ?it/s]
```


Data Analysis- Analytical Tools and Methods Used

1. Importing Necessary Libraries

```
In [1]: #pandas and numpy for data manipulation
#matplotlib for visuals
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.tree import tree
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import xgboost as xgb
```

2. Importing Data

Let's begin the analysis by importing the data and understanding its characteristics.

```
In [2]: df = pd.read_csv('Suicides in India 2001-2012.csv')
df.head(10)
```

Out[2]:

	State	Year	Type_code	Type	Gender	Age_group	Total
0	A & N Islands	2001	Causes	Illness (Aids/STD)	Female	0-14	0
1	A & N Islands	2001	Causes	Bankruptcy or Sudden change in Economic	Female	0-14	0
2	A & N Islands	2001	Causes	Cancellation/Non-Settlement of Marriage	Female	0-14	0
3	A & N Islands	2001	Causes	Physical Abuse (Rape/Incest Etc.)	Female	0-14	0
4	A & N Islands	2001	Causes	Dowry Dispute	Female	0-14	0
5	A & N Islands	2001	Causes	Family Problems	Female	0-14	0
6	A & N Islands	2001	Causes	Ideological Causes/Hero Worshiping	Female	0-14	0
7	A & N Islands	2001	Causes	Other Prolonged Illness	Female	0-14	0
8	A & N Islands	2001	Causes	Property Dispute	Female	0-14	0
9	A & N Islands	2001	Causes	Fall in Social Reputation	Female	0-14	0

```
In [3]: df.shape
print('Number of Rows: ' + str(df.shape[0]) + '\nNumber of Columns: ' + str(df.shape[1]))
Number of Rows: 237519
Number of Columns: 7
```

```
In [4]: # checking for data info
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 237519 entries, 0 to 237518
Data columns (total 7 columns):
# Column Non-Null Count Dtype
---
0 State 237519 non-null object
1 Year 237519 non-null int64
2 Type_code 237519 non-null object
3 Type 237519 non-null object
4 Gender 237519 non-null object
5 Age_group 237519 non-null object
6 Total 237519 non-null int64
dtypes: int64(2), object(5)
memory usage: 12.7+ MB
```

```
In [5]: # describing data for better statistical analysis and for better understanding of data
df.describe()
```

Out[5]:

	Year	Total
count	237519.000000	237519.000000
mean	2006.500448	55.034477
std	3.452240	792.749038
min	2001.000000	0.000000
25%	2004.000000	0.000000
50%	2007.000000	0.000000
75%	2010.000000	6.000000
max	2012.000000	62343.000000

3. Data Cleaning

Usually, data comes with many impurities such as missing values, duplicated columns or rows, and datatype errors etc. Therefore, it's important to clean the data properly before proceeding with further analysis to ensure higher accuracy.

Let's proceed with Data Cleaning.

```
In [6]: #checking for null values
df.isnull().sum()
```

Out[6]:

State	0
Year	0
Type_code	0
Type	0
Gender	0
Age_group	0
Total	0
dtype:	int64

Nice! There are no NaN values.

```
In [7]: # checking for duplicates
df.duplicated().sum()
```

Out[7]:

0

No Duplicates as well!

```
In [8]: # checking for column names
df.columns
```

Out[8]: Index(['State', 'Year', 'Type_code', 'Type', 'Gender', 'Age_group', 'Total'], dtype='object')

• Analytical Tools and Methods Used

```
In [9]: # checking for unique values in column 'State'  
df.State.unique()
```

```
Out[9]: array(['A & N Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam',  
             'Bihar', 'Chandigarh', 'Chhattisgarh', 'D & N Haveli',  
             'Daman & Diu', 'Delhi (Ut)', 'Goa', 'Gujarat', 'Haryana',  
             'Himachal Pradesh', 'Jammu & Kashmir', 'Jharkhand', 'Karnataka',  
             'Kerala', 'Lakshadweep', 'Madhya Pradesh', 'Maharashtra',  
             'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha',  
             'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu',  
             'Total (All India)', 'Total (States)', 'Total (Uts)', 'Tripura',  
             'Uttar Pradesh', 'Uttarakhand', 'West Bengal'], dtype=object)
```

```
In [10]: # checking how many times a state name is repeated  
df.State.value_counts()
```

```
Out[10]: State  
Madhya Pradesh      6792  
Maharashtra         6792  
Karnataka           6792  
Odisha              6791  
Andhra Pradesh      6791  
Rajasthan           6791  
Bihar               6790  
Chhattisgarh        6790  
Haryana             6790  
Kerala              6788  
Uttar Pradesh       6787  
Assam               6786  
Tamil Nadu          6786  
Gujarat             6786  
Jharkhand           6785  
Tripura             6782  
Delhi (Ut)          6782  
West Bengal         6780  
Punjab              6779  
Himachal Pradesh    6774  
Jammu & Kashmir      6761  
Goa                 6759  
Uttarakhand         6758  
Sikkim              6742  
Mizoram             6737  
Meghalaya           6733  
Puducherry          6730  
Chandigarh          6717
```

```
A & N Islands        6712  
Daman & Diu          6710  
Arunachal Pradesh   6707  
Nagaland             6705  
D & N Haveli         6704  
Manipur              6700  
Lakshadweep         6674  
Total (All India)    312  
Total (States)       312  
Total (Uts)          312  
Name: count, dtype: int64
```

There are some Rows named 'Total (All India)', 'Total (States)', 'Total (Uts)' which are not necessary, so let's just remove them.

```
In [11]: # Removing unnecessary rows from data  
df = df.query('State != ("Total (All India)", "Total (States)", "Total (Uts)")')  
df.shape
```

```
Out[11]: (236583, 7)
```

Now dataframe doesn't contain any rows with state name as 'Total (All India)', 'Total (States)', 'Total (Uts)'.

```
In [12]: df.State.unique()
```

```
Out[12]: array(['A & N Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam',  
             'Bihar', 'Chandigarh', 'Chhattisgarh', 'D & N Haveli',  
             'Daman & Diu', 'Delhi (Ut)', 'Goa', 'Gujarat', 'Haryana',  
             'Himachal Pradesh', 'Jammu & Kashmir', 'Jharkhand', 'Karnataka',  
             'Kerala', 'Lakshadweep', 'Madhya Pradesh', 'Maharashtra',  
             'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha',  
             'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu',  
             'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal'],  
             dtype=object)
```

```
In [13]: # Removing rows with zero total suicides because they do not add any value to the analysis and are therefore useless  
df = df.query('Total != 0')  
df.shape
```

```
Out[13]: (101110, 7)
```

```
In [14]: df.Total.value_counts()
```

```
Out[14]: Total  
1      16038  
2      9939  
3      6697  
4      5110  
5      4173
```

...

```
1774      1  
1332      1  
732       1  
1548      1  
2658      1
```

Name: count, Length: 1763, dtype: int64

4. Data Analysis

Now the data is almost cleaned and ready for further analysis.

```
In [15]: # checking how many times each unique value occurs in the Type_code column  
df.Type_code.value_counts()
```

```
Out[15]: Type_code  
Causes      39531  
Means_adopted 29337  
Professional_Profile 23291  
Education_Status 5602  
Social_Status 3349  
Name: count, dtype: int64
```

● Data Analysis- Key Findings

9.5 Model Training with XGBoost

```
In [63]: # Grid search for XGBoost
grid_xgb = GridSearchCV(xgb.XGBClassifier(), param_grid_xgb, cv=5, n_jobs=-1, verbose=1)
grid_xgb.fit(X_train, y_train)
```

Fitting 5 folds for each of 72 candidates, totalling 360 fits

```
Out[63]:
> GridSearchCV
> estimator: XGBClassifier
> XGBClassifier
```

```
In [64]: # Best parameters and score for XGBoost
best_xgb = grid_xgb.best_estimator_
print("Best parameters for XGBoost:", grid_xgb.best_params_)
print("Best cross-validation score for XGBoost:", grid_xgb.best_score_)

Best parameters for XGBoost: {'colsample_bytree': 1.0, 'learning_rate': 0.2, 'max_depth': 7, 'n_estimators': 200, 'subsample': 0.8}
Best cross-validation score for XGBoost: 0.812622748644588
```

```
In [65]: # Train the best XGBoost model
best_xgb.fit(X_train, y_train)
```

```
Out[65]:
> XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
colsample_bylevel=None, colsample_bynode=None,
colsample_bytree=1.0, device=None, early_stopping_rounds=None,
enable_categorical=False, eval_metric=None, feature_types=None,
gamma=None, grow_policy=None, importance_type=None,
interaction_constraints=None, learning_rate=0.2, max_bin=None,
max_cat_threshold=None, max_cat_to_onehot=None,
max_delta_step=None, max_depth=7, max_leaves=None,
min_child_weight=None, missing=nan, monotone_constraints=None,
multi_strategy=None, n_estimators=200, n_jobs=None,
num_parallel_tree=None, objective='multi:softprob', ...)
```

```
In [66]: # Evaluate the best XGBoost model
y_pred_xgb = best_xgb.predict(X_test)
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
print(f"Best XGBoost Accuracy: {accuracy_xgb:.2f}")

Best XGBoost Accuracy: 0.82
```

```
In [67]: print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

```
Out[67]:
Best XGBoost Classification Report:
      precision    recall  f1-score   support\n
0       0.87      0.84      0.85       3256\n
1       0.92      0.93      0.92       297\n
2       0.81      0.81      0.81      3033\n
accuracy   0.82\nmacro avg   0.82      0.82      0.82      3033\nweighted avg 0.82      0.82      0.82      3033
```

```
In [68]: print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

```
Out[68]:
Best XGBoost Classification Report:
      precision    recall  f1-score   support\n
0       0.87      0.84      0.85       3256\n
1       0.92      0.93      0.92       297\n
2       0.81      0.81      0.81      3033\n
accuracy   0.82\nmacro avg   0.82      0.82      0.82      3033\nweighted avg 0.82      0.82      0.82      3033
```

Lastly Generate a pandas profiling for even better visualization

```
In [74]: from pandas_profiling import ProfileReport
```

```
# Generate the report
profile = ProfileReport(df, title="Suicides in India Dataset Profiling Report")
profile.to_file("suicides_in_india_profile_report.html")
```

```
# Display the report
profile.to_notebook_iframe()
```

```
Summarize dataset: 0% | 0/5 [00:00<, ?it/s]
Generate report structure: 0% | 0/1 [00:00<, ?it/s]
Render HTML: 0% | 0/1 [00:00<, ?it/s]
Export report to file: 0% | 0/1 [00:00<, ?it/s]
```

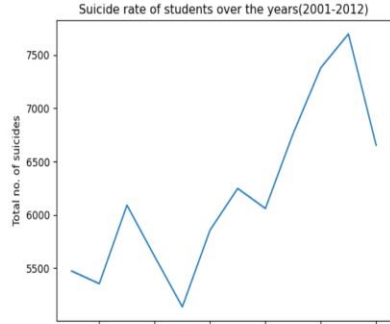
Suicides in India Dataset Profiling Report

Overview Variables Interactions Correlations Missing values Sample

● Data Analysis- Insights Derived

6.2.1 Increasing no. of suicides among Students over the years

```
In [38]: # plotting a line graph No. of suicides vs year
student.groupby('Year')['Total'].sum().plot(title='Suicide rate of students over the years(2001-2012)')
plt.xlabel('Years')
plt.ylabel('Total no. of suicides')
plt.show()
```



8. Means Adopted for Suicide

It is also crucial to identify the methods people are using to commit suicide in order to find solutions.

```
In [43]: df.Type_code.value_counts()
```

```
Out[43]:
Type_code
Causes          39531
Means_adopted    29337
Professional_Profile  23291
Education_Status  5602
Social_Status    3349
Name: count, dtype: int64
```

```
In [44]: # creating a dataframe which contains only means adopted by people to commit suicide
means_adopted = df.query('Type_code=="Means_adopted").copy().reset_index(drop=True)

#dropping Type_code column
means_adopted = means_adopted.drop(columns=['Type_code'],axis=1)
means_adopted.head()
```

```
Out[44]:
   State  Year  Type  Gender  Age_group  Total
0  A & N Islands  2001  By Hanging  Female  0-14  2
1  A & N Islands  2001  By Hanging  Male  0-14  2
2  A & N Islands  2001  By Fire/Self Immolation  Female  15-29  2
3  A & N Islands  2001  By Consuming Other Poison  Female  15-29  9
4  A & N Islands  2001  By Hanging  Female  15-29  20
```

```
In [45]: means_adopted.Type.value_counts()
```

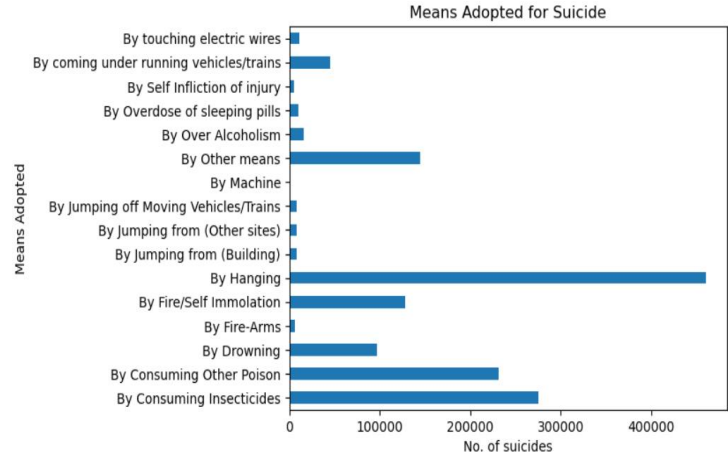
```
Out[45]:
Type
By Hanging          3514
By Consuming Other Poison  2952
By Drowning          2609
By Consuming Insecticides  2608
By Fire/Self Immolation  2582
By Other means (please specify)  2415
By coming under running vehicles/trains  1782
```

```
By touching electric wires          1483
By Jumping from (Other sites)        1384
By Jumping from (Building)           1325
By Overdose of sleeping pills        1272
By Fire-Arms                         1236
By Over Alcoholism                   1228
By Jumping off Moving Vehicles/Trains 1137
By Self Infliction of Injury         1093
By Machine                           504
By Other means                       213
Name: count, dtype: int64
```

```
In [46]: # there are two unique values that mean the same, so let's combine them
means_adopted = means_adopted.replace(['By Other means (please specify)'], ['By Other means'])
means_adopted.Type.value_counts()
```

```
Out[46]:
Type
By Hanging          3514
By Consuming Other Poison  2952
By Other means      2608
By Drowning          2609
By Consuming Insecticides  2608
By Fire/Self Immolation  2582
By coming under running vehicles/trains  1782
By touching electric wires  1483
By Jumping from (Other sites)  1384
By Jumping from (Building)     1325
By Overdose of sleeping pills  1272
By Fire-Arms                 1236
By Over Alcoholism           1228
By Jumping off Moving Vehicles/Trains 1137
By Self Infliction of Injury  1093
By Machine                  504
Name: count, dtype: int64
```

```
In [47]: # plotting a bar graph for better understanding
means_adopted.groupby('Type')['Total'].sum().plot(kind='barh', title='Means Adopted for Suicide')
plt.xlabel('No. of suicides')
plt.ylabel('Means Adopted')
plt.show()
```



7. Social Status

```
In [39]: df.Type_code.value_counts()
```

```
Out[39]: Type_code
Causes          39531
Means_adopted    29337
Professional_Profile 23291
Education_Status  5602
Social_Status    3349
Name: count, dtype: int64
```

```
In [40]: # creating a dataframe consisting only social status of the people who committed suicides
social_status = df.query('Type_code=="Social_Status").copy().reset_index(drop=True)
```

```
# since Type_code has only 1 value so it's better to delete it
social_status = social_status.drop(columns=['Type_code'],axis=1)
social_status.head()
```

```
Out[40]:
```

	State	Year	Type	Gender	Age_group	Total
0	A & N Islands	2001	Married	Female	0-100+	38
1	A & N Islands	2001	Never Married	Female	0-100+	12
2	A & N Islands	2001	Married	Male	0-100+	55
3	A & N Islands	2001	Never Married	Male	0-100+	24
4	A & N Islands	2002	Never Married	Female	0-100+	17

```
In [41]: social_status.Age_group.value_counts()
```

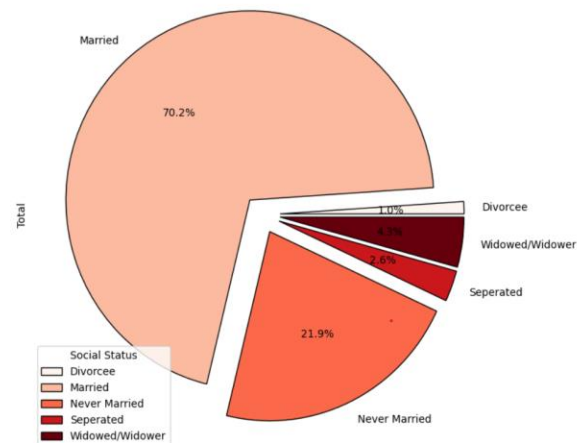
```
Out[41]: Age_group
0-100+    3349
Name: count, dtype: int64
```

```
In [42]: # plotting a pie chart of the social status of the people who committed suicide
```

```
social_status.groupby('Type')['Total'].sum().plot.pie(y='Total',
    figsize=(8,8),
    autopct='%1.1f%%',
    title='Social Status of People who committed suicide(2001-2012)',
    cmap = 'Reds',
    explode = (0.1,0.1,0.1,0.1,0.1),
    wedgeprops = {"edgecolor":"black",
        'linewidth': 1,
        'antialiased': True})

plt.legend(title='Social Status')
plt.show()
```

Social Status of People who committed suicide(2001-2012)



Hypothesis Development-Formulated Hypothesis

9.4 Model Training

```
# Grid search for GBM
grid_gbm = GridSearchCV(HistGradientBoostingClassifier(), param_grid_gbm, cv=5, n_jobs=-1, verbose=1)
grid_gbm.fit(X_train, y_train)
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

```
>
  GridSearchCV
  estimator: HistGradientBoostingClassifier
    HistGradientBoostingClassifier
```

```
# Best parameters and score for GBM
best_gbm = grid_gbm.best_estimator_
print("Best parameters for GBM:", grid_gbm.best_params_)
print("Best cross-validation score for GBM:", grid_gbm.best_score_)
```

Best parameters for GBM: {'learning_rate': 0.1, 'max_depth': 7, 'max_iter': 100, 'max_leaf_nodes': 63}
Best cross-validation score for GBM: 0.6612031329958181

```
# Train the GBM model
gbm = HistGradientBoostingClassifier()
gbm.fit(X_train, y_train)
```

```
> HistGradientBoostingClassifier
HistGradientBoostingClassifier()
```

```
# Train the best GBM model
best_gbm.fit(X_train, y_train)
```

```
>
  HistGradientBoostingClassifier
  HistGradientBoostingClassifier(max_depth=7, max_leaf_nodes=63)
```

```
# Evaluate the best GBM model
y_pred_gbm = best_gbm.predict(X_test)
accuracy_gbm = accuracy_score(y_test, y_pred_gbm)
print(f"Best GBM Accuracy: {accuracy_gbm:.2f}")

Best GBM Accuracy: 0.68
```

The accuracy of the model is 68 % let's train it further

```
print("Best GBM Classification Report:")
print(classification_report(y_test, y_pred_gbm))
```

```
Best GBM Classification Report:
      precision    recall  f1-score   support

0       0.70      0.97      0.82     16344
1       0.52      0.24      0.33      7425
2       0.72      0.67      0.69      3356
3       0.80      0.86      0.83       257
4       0.53      0.07      0.13      2337
5       0.68      0.45      0.54       557
6       0.25      0.94      0.40        17

 accuracy
macro avg      0.60      0.60      0.53     30333
weighted avg      0.65      0.68      0.63     30333
```

```
print("Best GBM Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_gbm))
```

```
Best GBM Confusion Matrix:
[[15900  368   45    5    8    5   13]
 [ 5355 1814  156    0   85    2   13]
 [   479   486 2257    7   54   73    0]
 [     2    0   10  254    0   30    1]
 [   895   809   432    5  169    6   21]
 [    11    5  243   46    0  252    0]
 [     0    0    0    1    0    0   16]]
```

```
# Define the parameter grid for XGBoost
param_grid_xgb = {
    'learning_rate': [0.01, 0.1, 0.2],
    'n_estimators': [100, 200],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
```

9.5 Model Training with XGBoost

```
# Grid search for XGBoost
grid_xgb = GridSearchCV(xgb.XGBClassifier(), param_grid_xgb, cv=5, n_jobs=-1, verbose=1)
grid_xgb.fit(X_train, y_train)
```

Fitting 5 folds for each of 72 candidates, totalling 360 fits

```
>
  GridSearchCV
  estimator: XGBClassifier
    XGBClassifier
```

```
# Best parameters and score for XGBoost
best_xgb = grid_xgb.best_estimator_
print("Best parameters for XGBoost:", grid_xgb.best_params_)
print("Best cross-validation score for XGBoost:", grid_xgb.best_score_)
```

Best parameters for XGBoost: {'colsample_bytree': 1.0, 'learning_rate': 0.2, 'max_depth': 7, 'n_estimators': 200, 'subsample': 0.8}
Best cross-validation score for XGBoost: 0.812622748644588

```
# Train the best XGBoost model
best_xgb.fit(X_train, y_train)
```

```
>
  XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
  colsample_bylevel=None, colsample_bynode=None,
  colsample_bytree=1.0, device=None, early_stopping_rounds=None,
  enable_categorical=False, eval_metric=None, feature_types=None,
  gamma=None, grow_policy=None, importance_type=None,
  interaction_constraints=None, learning_rate=0.2, max_bin=None,
  max_cat_threshold=None, max_cat_to_onehot=None,
  max_delta_step=None, max_depth=7, max_leaves=None,
  min_child_weight=None, missing=nan, monotone_constraints=None,
  multi_strategy=None, n_estimators=200, n_jobs=None,
  num_parallel_tree=None, objective='multi:softprob', ...)
```

Hypothesis Development-Formulated Hypothesis

● Data Transformation Techniques

```
# Evaluate the best XGBoost model
y_pred_xgb = best_xgb.predict(X_test)
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
print(f"Best XGBoost Accuracy: {accuracy_xgb:.2f}")
```

Best XGBoost Accuracy: 0.82

```
print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

Best XGBoost Classification Report:

	precision	recall	f1-score	support	0	0.88	0.93	0.91	16344	1	0.71	0.68	0.70	7425	2	0.82	0.87	0.84	3356	3	0.93	0.91	0.92	297	4					
0.62	0.41	0.50	2337	5	0.78	0.74	0.76	557	6	1.00	0.94	0.97	17	n	n	accuracy	0.82	0.87	0.84	3356	n	0.82	0.78	0.80	30333	n	weighted avg	0.81	0.82	
0.81	30333	n																												

```
print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

Best XGBoost Classification Report:

	precision	recall	f1-score	support	0	0.88	0.93	0.91	16344	1	0.71	0.68	0.70	7425	2	0.82	0.87	0.84	3356	3	0.93	0.91	0.92	297	4					
0.62	0.41	0.50	2337	5	0.78	0.74	0.76	557	6	1.00	0.94	0.97	17	n	n	accuracy	0.82	0.87	0.84	3356	n	0.82	0.78	0.80	30333	n	weighted avg	0.81	0.82	
0.81	30333	n																												

Lastly Generate a pandas profiling for even better visualization

```
from pandas_profiling import ProfileReport
```

```
# Generate the report
```

```
profile = ProfileReport(df, title="Suicides in India Dataset Profiling Report")
profile.to_file("suicides_in_india_profile_report.html")
```

```
# Display the report
```

```
profile.to_notebook_iframe()
```

```
Summarize dataset: 0% | 0/5 [00:00<?, ?it/s]
Generate report structure: 0% | 0/1 [00:00<?, ?it/s]
Render HTML: 0% | 0/1 [00:00<?, ?it/s]
Export report to file: 0% | 0/1 [00:00<?, ?it/s]
```

- **Hypothesis Development-Rationale Behind the Hypothesis**

- **Hypothesis Development-Method for Testing the Hypothesis**

```
# Evaluate the best XGBoost model
y_pred_xgb = best_xgb.predict(X_test)
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
print(f"Best XGBoost Accuracy: {accuracy_xgb:.2f}")
```

Best XGBoost Accuracy: 0.82

```
print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

Best XGBoost Classification Report:

[illegible]

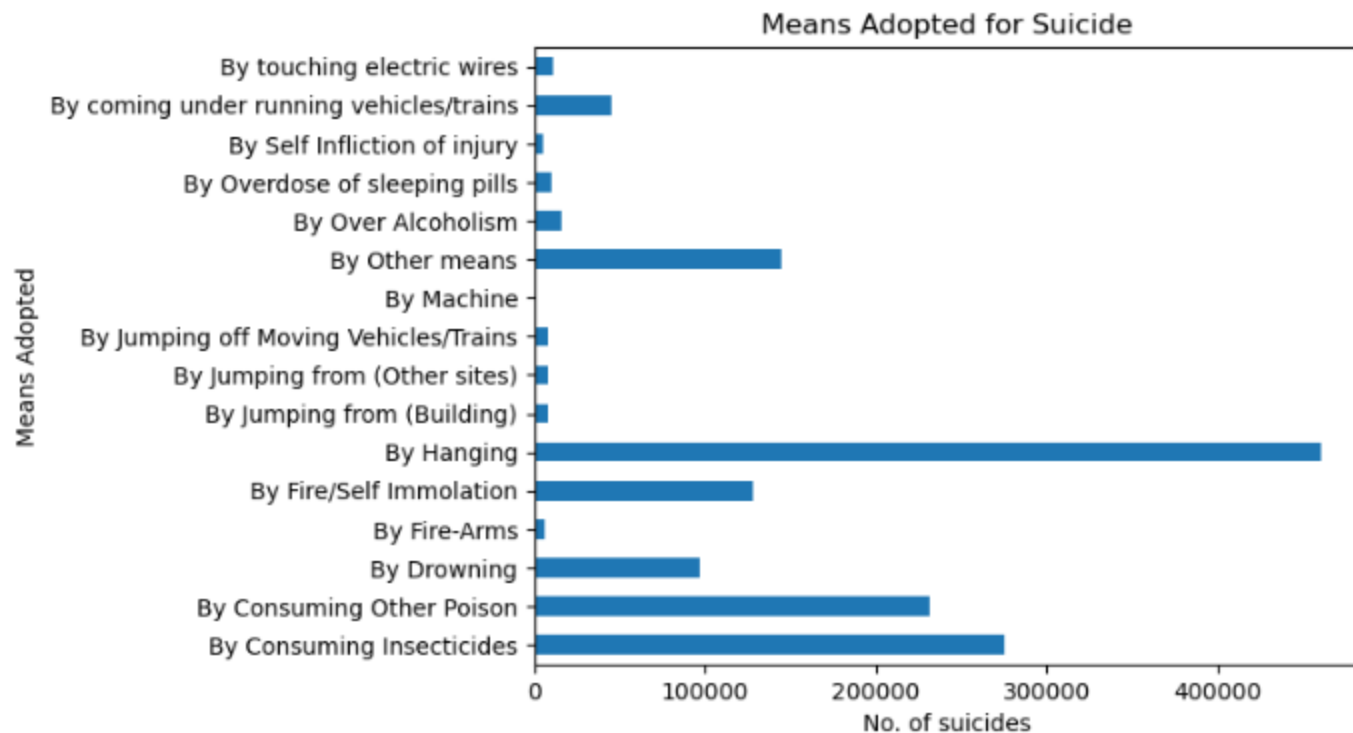
```
print("Best XGBoost Classification Report:")
classification_report(y_test, y_pred_xgb)
```

Best XGBoost Classification Report:

[illegible]

Solution Design- Proposed Solution

```
: # plotting a bar graph for better understanding
means_adopted.groupby('Type')['Total'].sum().plot(kind='barh',title='Means Adopted for Suicide')
plt.xlabel('No. of suicides')
plt.ylabel('Means Adopted')
plt.show()
```



Solution Design- Implementation Plan

9.3 Splitting Data

```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(features, target, test_size=0.3, random_state=42)
```

```
# Define the parameter grid for GBM
param_grid_gbm = {
    'learning_rate': [0.01, 0.1, 0.2],
    'max_iter': [100, 200],
    'max_leaf_nodes': [31, 63],
    'max_depth': [3, 5, 7]
}
```

9.4 Model Training

```
# Grid search for GBM
grid_gbm = GridSearchCV(HistGradientBoostingClassifier(), param_grid_gbm, cv=5, n_jobs=-1, verbose=1)
grid_gbm.fit(X_train, y_train)
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

```
> GridSearchCV
> estimator: HistGradientBoostingClassifier
  > HistGradientBoostingClassifier
```

```
# Best parameters and score for GBM
best_gbm = grid_gbm.best_estimator_
print("Best parameters for GBM:", grid_gbm.best_params_)
print("Best cross-validation score for GBM:", grid_gbm.best_score_)
```

Best parameters for GBM: {'learning_rate': 0.1, 'max_depth': 7, 'max_iter': 100, 'max_leaf_nodes': 63}
Best cross-validation score for GBM: 0.6612031329958181

```
# Train the GBM model
gbm = HistGradientBoostingClassifier()
gbm.fit(X_train, y_train)
```

```
> HistGradientBoostingClassifier
HistGradientBoostingClassifier()
```

```
# Train the best GBM model
best_gbm.fit(X_train, y_train)
```

```
> HistGradientBoostingClassifier
HistGradientBoostingClassifier(max_depth=7, max_leaf_nodes=63)
```

```
# Evaluate the best GBM model
y_pred_gbm = best_gbm.predict(X_test)
accuracy_gbm = accuracy_score(y_test, y_pred_gbm)
print(f"Best GBM Accuracy: {accuracy_gbm:.2f}")
```

Best GBM Accuracy: 0.68

The accuracy of the model is 68 % let's train it further

```
print("Best GBM Classification Report:")
print(classification_report(y_test, y_pred_gbm))
```

```
Best GBM Classification Report:
              precision    recall  f1-score   support

     0       0.70      0.97      0.82     16344
     1       0.52      0.24      0.33      7425
     2       0.72      0.67      0.69      3356
     3       0.80      0.86      0.83       297
     4       0.53      0.07      0.13      2337
     5       0.68      0.45      0.54       557
     6       0.25      0.94      0.40        17

 accuracy          0.68      30333
 macro avg         0.60      0.60      0.53      30333
 weighted avg      0.65      0.68      0.63      30333
```

```
print("Best GBM Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_gbm))
```

```
Best GBM Confusion Matrix:
[[15900  368   45    5    8    5   13]
 [ 5355 1814  156    0   85    2   13]
 [  479   486 2257    7   54   73    0]
 [    2    0   10  254    0   30    1]
 [  895   809  432    5  169    6   21]
 [   11    5  243   46    0  252    0]
 [    0    0    0    1    0    0   16]]
```

```
# Define the parameter grid for XGBoost
param_grid_xgb = {
    'learning_rate': [0.01, 0.1, 0.2],
    'n_estimators': [100, 200],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
```

9.5 Model Training with XGBoost

```
# Grid search for XGBoost
grid_xgb = GridSearchCV(xgb.XGBClassifier(), param_grid_xgb, cv=5, n_jobs=-1, verbose=1)
grid_xgb.fit(X_train, y_train)
```

Fitting 5 folds for each of 72 candidates, totalling 360 fits

```
> GridSearchCV
> estimator: XGBClassifier
  > XGBClassifier
```

```
# Best parameters and score for XGBoost
best_xgb = grid_xgb.best_estimator_
print("Best parameters for XGBoost:", grid_xgb.best_params_)
print("Best cross-validation score for XGBoost:", grid_xgb.best_score_)
```

Best parameters for XGBoost: {'colsample_bytree': 1.0, 'learning_rate': 0.2, 'max_depth': 7, 'n_estimators': 200, 'subsample': 0.8}
Best cross-validation score for XGBoost: 0.812622748644588

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bynode=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None, feature_types=None,
               gamma=None, grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=0.2, max_bin=None,
               max_cat_threshold=None, max_cat_to_onehot=None,
               max_delta_step=None, max_depth=7, max_leaves=None,
               min_child_weight=None, missing=None, monotone_constraints=None,
               multi_strategy=None, n_estimators=200, n_jobs=None,
               num_parallel_tree=None, objective='multisoftprob', ...)
```

Best XGBoost Accuracy: 0.82

```

Best XGBoost Classification Report:
'
precision    recall  f1-score   support\n\n      0            0.88       0.93       0.91     16344\n1              1            0.71       0.68       0.70     7425\n2              2            0.82       0.87       0.84     3356\n3              3            0.93       0.91       0.92     297\n4              4            0.82       0.81       0.82     30333\n'
```

Test XGBoost Classification Report:																												
				precision	recall	f1-score	support\n	0	0.88	0.93	0.91	16344\n	1	0.71	0.68	0.70	7425\n	2	0.82	0.87	0.84	3356\n	3	0.93	0.91	0.92	297\n	4
0.62	0.41	0.50	2337\n	5	0.78	0.74	0.76	557\n	6	1.00	0.94	0.97	17\n	accuracy	0.82	0.87	0.84	30333\n	macro avg	0.82	0.78	0.80	30333\n	weighted avg	0.81	0.82		

```
from pandas_profiling import ProfileReport

# Generate the report
profile = ProfileReport(df, title="Suicides in India Dataset Profiling Report")
profile.to_file("suicides_in_india_profile_report.html")

# Display the report
profile.to_notebook_iframe()
```

```
Summarize dataset: 0% | 0/5 [00:00<, ?it/s]
Generate report structure: 0% | 0/1 [00:00<, ?it/s]
Render HTML: 0% | 0/1 [00:00<, ?it/s]
Export report to file: 0% | 0/1 [00:00<, ?it/s]
```

Solution Design- Alignment with SDGs

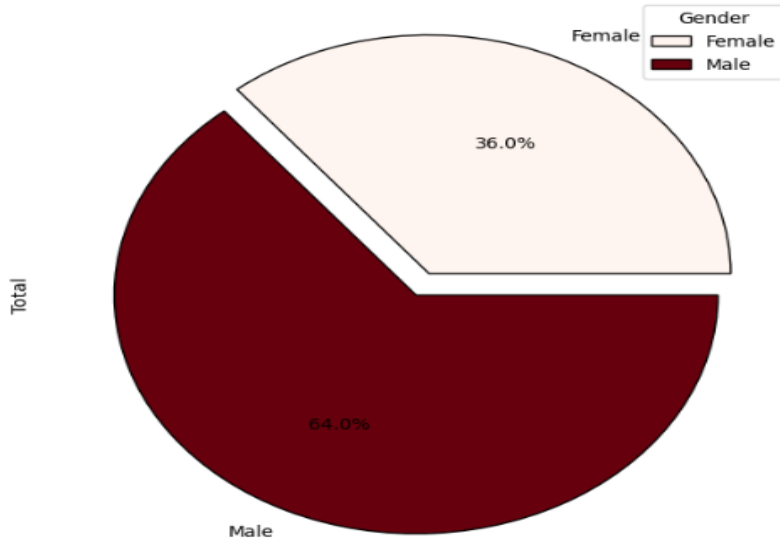
Visualization- Data Visualization Techniques Used

4.2 Suicides among Males and Females (2001-2012)

```
: # plotting a pie chart for suicides among Males and Females (2001-2012)
ax = df.groupby('Gender')['Total'] \
    .sum().plot.pie(y = 'Total',
                    figsize=(7,7),
                    autopct='%1.1f%%',
                    cmap = 'Reds',
                    title='Suicides among Males and Females(2001-2012)',
                    explode = (0.1,0),
                    wedgeprops= {"edgecolor": "black",
                                'linewidth': 1,
                                'antialiased': True})

plt.legend(title='Gender')
plt.show()
```

Suicides among Males and Females(2001-2012)



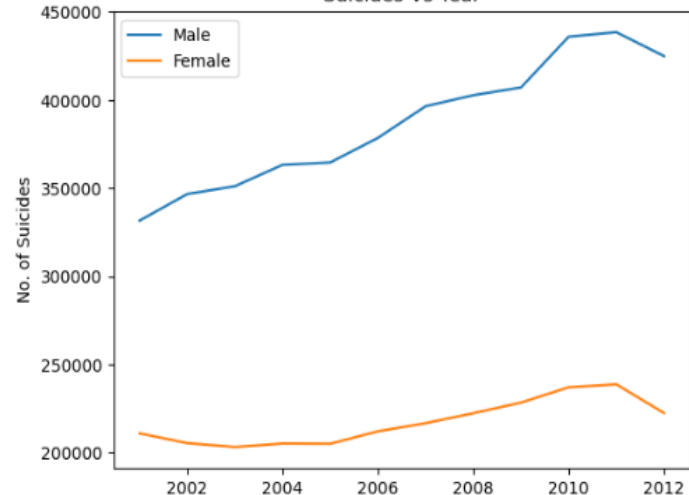
4.2.1 Variation of No. of suicides among Males and Females over the years

```
]: # creating a dataframe for males
male = df.query('Gender == "Male"')

# creating a dataframe for females
female = df.query('Gender == "Female"')

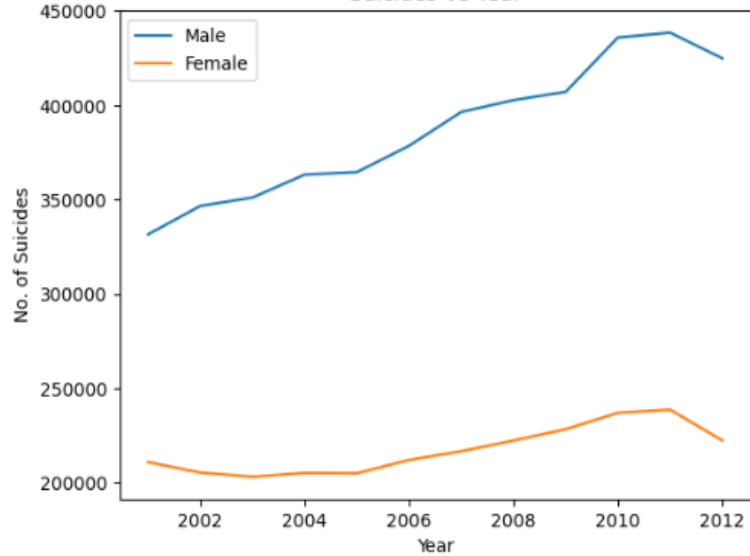
# plotting a graph for observing the variation with year
ax = male.groupby('Year')['Total'].sum().plot(title = 'Suicides Vs Year', label = 'Male')
ax = female.groupby('Year')['Total'].sum().plot(label = 'Female')
plt.xlabel('Year')
plt.ylabel('No. of Suicides')
plt.legend()
plt.show()
```

Suicides Vs Year

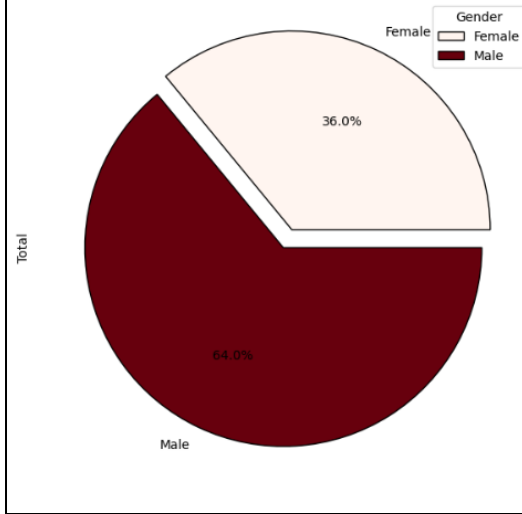


Visualization- Charts, Graphs, Maps, etc.

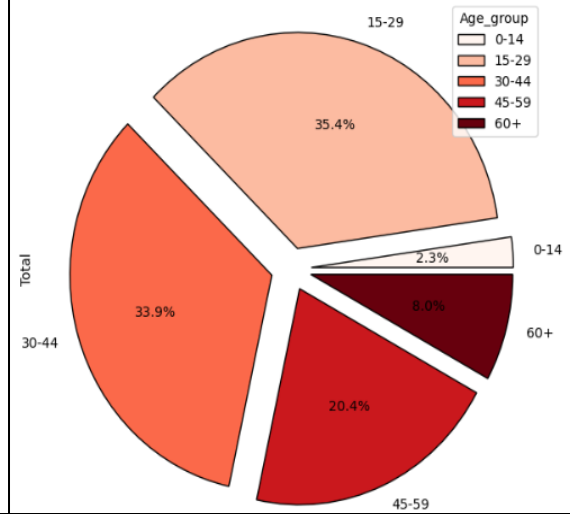
Suicides Vs Year



Suicides among Males and Females(2001-2012)

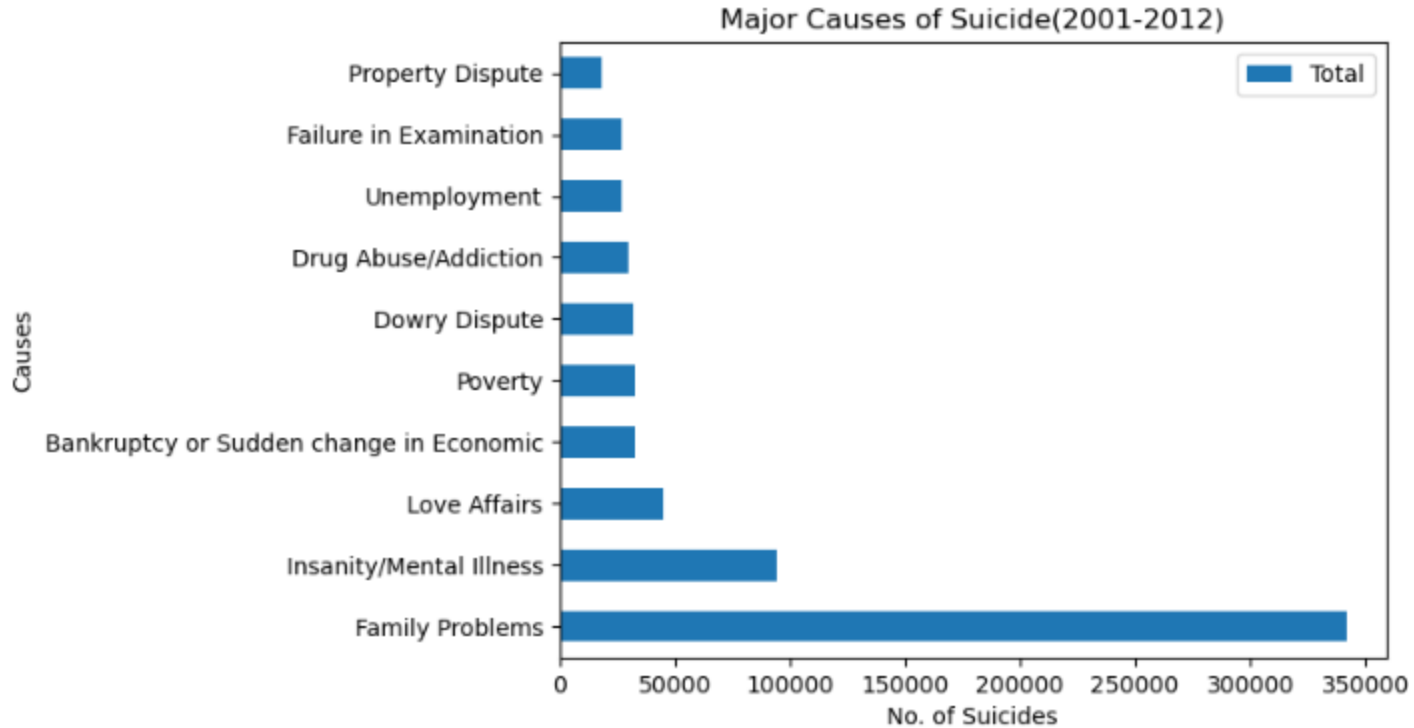


Age_group of People who committed Suicide(2001-2012)



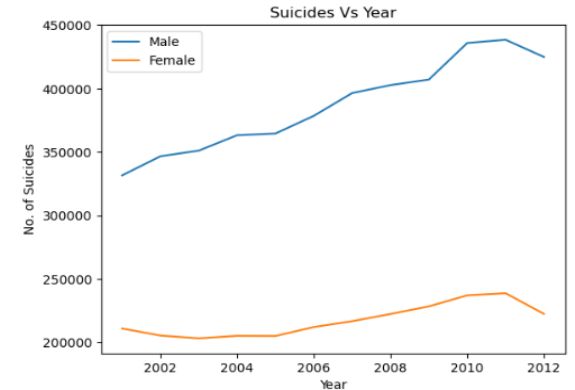
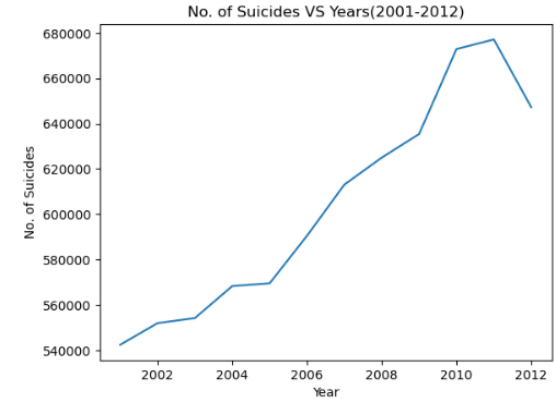
Visualization- Key Visual Insights.

.



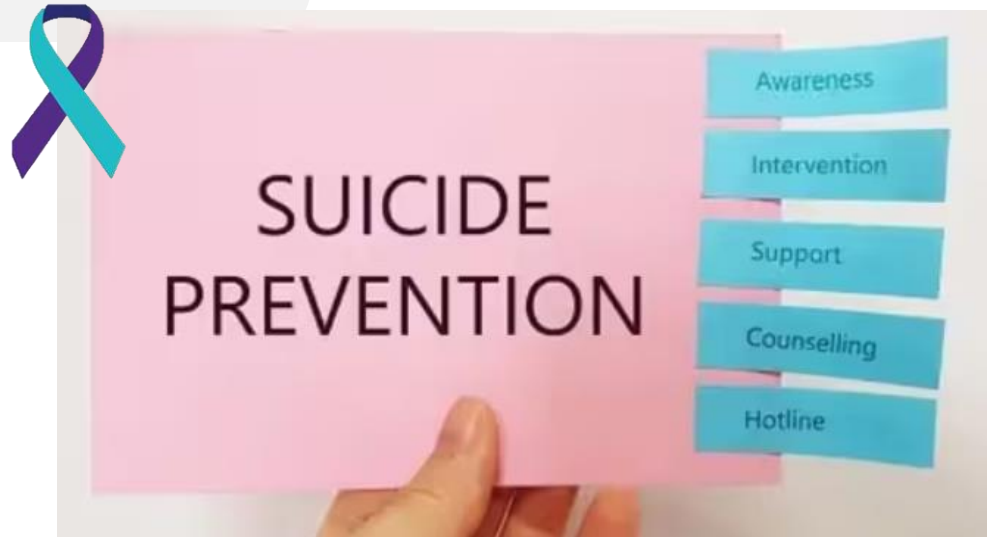
Conclusion- Summary of Findings

- Analyzed suicide rates in India from 2001 to 2012.
- Identified key demographic, geographic, and socio-economic factors.
- Developed a predictive model with an accuracy of 0.82.
- Successfully forecasted future suicide trends and identified potential risk periods.



Conclusion- Future Work

- Refine the predictive model by incorporating more recent data.
- Explore additional factors influencing suicide rates.
- Enhance model accuracy and provide deeper insights for more effective prevention strategies.



Conclusion- Impact of Proposed Solution

- Provides evidence-based strategies for policymakers and public health officials.
- Allows for targeted interventions to address high-risk periods and demographic groups.
- Potentially reduces suicide rates and improves public health outcomes.



References

- **Data Sources**
- **Tools and Software
Used**
- **Additional References**